Task 1 - Web App QA & Debug Report

Candidate: Aryan **Role:** Automation & QA Developer take-home **Date:** 29 Sep 2026

1. Scope and method

The repository I cloned (realworld-apps/realworld) is the **spec and test-suite hub**, not a runnable app. I therefore ran a real implementation from the ecosystem: the **gothinkster Django REST backend** (Django 1.11, Python 3.8, SQLite), and tested it with the official spec suite (specs/api/bruno: 143 requests in 11 folders) plus manual curl checks of register, login, article create/edit/delete, comments, follow, feed, favorites and profile update. Every bug below was reproduced with curl (task1/repro.sh). **Limits:** API layer only; I did not test a browser UI, so UX and accessibility issues are not covered. I also ran Django 1.11 instead of the pinned 1.10.5 because 1.10.5 crashes on Python 3.8.

Spec-suite result: 49 of 143 requests passed fully; 94 had at least one failing assertion. Many failures cascade from a few causes (status codes, error format, null vs empty string), so the table groups by cause and ranks by user impact.

2. Bug table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	**Any logged-in user can edit another user's article**	1. Register users A and B. 2. A: POST /api/articles. 3. B: PUT /api/articles/{A's slug} with body text "HACKED".	Expected: 403, article unchanged. Actual: 200 OK, B's text saved (verified with curl).	**Critical**	ArticleViewSet.update() loads the article by slug and saves it; it never compares the article author with request.user. Only IsAuthenticatedOrReadOnly is applied.
2	**Any logged-in user can delete anyone's comment**	1. A creates article and comment. 2. B: DELETE /api/articles/{slug}/comments/{id}.	Expected: 403, comment survives. Actual: 204, comment deleted (spec tests errors-authorization 07-08 fail).	**Critical**	CommentsDestroyAPIView.destroy() does Comment.objects.get(pk=...).delete() with no author check; the slug in the URL is ignored.
3	**'Your Feed' endpoint is unreachable**	1. Log in. 2. GET /api/articles/feed with a valid token.	Expected: 200 with followed users' articles (empty list for a new user). Actual: 404 "An article with this slug does not exist."	**High**	The router route articles/{slug} is registered before the explicit articles/feed route, so "feed" is treated as a slug (URL order in articles/urls.py).
4	**Articles cannot be deleted**	1. Create an article. 2. DELETE /api/articles/{slug} with a valid token (also with trailing slash).	Expected: 204. Actual: 405 Method Not Allowed (404 with trailing slash). Content can never be removed.	**High**	ArticleViewSet mixes in Create, List and Retrieve only; DestroyModelMixin / destroy() is missing.
5	**Password can be changed to 1 character, empty or null**	1. Log in. 2. PUT /api/user with password "1" (also "" and null).	Expected: 422 (same 8-character rule as registration). Actual: 200 OK, weak password saved.	**High**	Length rules exist only on the registration serializer; the update path has no min_length / allow_blank checks.
6	**Hard-coded SECRET_KEY, DEBUG=True, ALLOWED_HOSTS=[]**	1. Open conduit/settings.py lines 23-28.	Expected: secrets from environment, DEBUG off. Actual: literal key committed to the repo. JWTs can be forged by anyone who reads the code; DEBUG leaks stack traces.	**High**	Project template defaults left in place; no environment-based configuration.
7	**Unauthenticated requests return 403, not 401, with an odd error shape**	1. GET /api/user with no Authorization header (also with 'Token abc').	Expected: 401 and {"errors":{...}}. Actual: 403 and {"user":{"detail":"..."}}. A frontend cannot tell 'session expired' from 'not allowed'.	**Medium**	Suspected: the custom JWT authentication class has no authenticate_header(), so DRF turns 401 into 403; the renderer wraps DRF's detail under the resource key.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
8	**Wrong status codes and error text for validation, duplicates, login**	1. POST /api/users with empty username. 2. Register the same username twice. 3. Log in with a wrong password.	Expected: 422 "can't be blank", 409 for duplicates, 401 for bad login. Actual: 400 with DRF wording ("This field may not be blank.").	**Medium**	Default DRF exception handling; no custom handler maps errors to the agreed API format. Unknown profile/article errors are strings, not arrays.
9	**User JSON: bio and image are "" or missing instead of null**	1. Register a user and inspect the JSON. 2. GET /api/user.	Expected: bio and image keys present, null when unset. Actual: register/login return only email, username, token; GET /api/user returns "". An empty image string shows a broken avatar.	**Medium**	Register/login serializers omit the profile fields; profile fields default to empty string instead of null.
10	**Comments on an unknown article return 200 with an empty list**	1. GET /api/articles/does-not-exist/comments.	Expected: 404. Actual: 200 {"comments": [], "commentsCount": 0}. Hides typos and stale links.	**Low**	The list view only filters by slug and never checks that the article exists (create() does).
11	**Script payload in title is stored and returned unsanitised**	1. POST /api/articles with a title containing an img tag with an onerror handler.	Expected: clear contract on output encoding. Actual: stored verbatim. Safe only if every frontend escapes output (informational).	**Low**	No server-side sanitisation and no documented encoding rule.

3. Root-cause analysis: bug #1 (editing someone else's article)

What happens. When user B sends PUT /api/articles/{slug} for an article written by user A, the API returns 200 OK and stores B's text; I confirmed this with curl. **Why.** In articles/views.py, ArticleViewSet.update() fetches the article by slug, builds a partial serializer and saves it. The only guard is the class-level permission IsAuthenticatedOrReadOnly, which asks "is someone logged in?" but never "does this article belong to them?". This is broken access control (IDOR); the same gap exists in CommentsDestroyAPIView (bug #2). **Impact.** Any registered user can deface or spam every article, which blocks production use. **Fix.** Add an ownership check that returns 403 when the article author is not the requesting user, ideally as a reusable permission class applied to article update/delete and comment delete. **Prevention.** The spec folder errors-authorization already tests this; run it in CI.

Suggested code

```
# articles/views.py, inside ArticleViewSet.update(), right after fetching the article
from rest_framework.exceptions import PermissionDenied

if serializer_instance.author != request.user.profile:
    raise PermissionDenied("You can only edit your own articles.")   # HTTP 403

# articles/views.py, inside CommentsDestroyAPIView.destroy(), before comment.delete()
if comment.author != request.user.profile:
    raise PermissionDenied("You can only delete your own comments.")
```

4. Recommended fix order

(1) Ownership checks on article update and comment delete. (2) Move SECRET_KEY and DEBUG to environment variables and rotate the key. (3) Add article DELETE. (4) Fix route order so /articles/feed works. (5) Password rules on update. (6) Standardise status codes and error format. (7) null instead of empty strings.